

OSSP_SIGNALS

1. SIGTERM

```
GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    printf("\nSIGTERM received!\n");
    printf("Signal number = %d\n", sig);
}

int main()
{
    signal(SIGTERM, handler);

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
SIGTERM received!
```

```
Signal number = 15
```

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
|
```

```
hemasai@Hemasai:~$ cd ~/OSSP_Assignment
hemasai@Hemasai:~/OSSP_Assignment$ pgrep Sigterm
629
hemasai@Hemasai:~/OSSP_Assignment$ kill -TERM 12345
-bash: kill: (12345) - No such process
hemasai@Hemasai:~/OSSP_Assignment$ pgrep Sigterm
629
hemasai@Hemasai:~/OSSP_Assignment$ kill -TERM 629
hemasai@Hemasai:~/OSSP_Assignment$ █
```

2.SIGCHLD

```
GNU nano 2.7.1                               Sigchild.c
#include <unistd.h>
#include <sys/wait.h>

void child_handler(int sig)
{
    printf("SIGCHLD received: Child has terminated\n");
    wait(NULL);
}

int main()
{
    signal(SIGCHLD, child_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child is running...\n");
        sleep(2);
        printf("Child is exiting...\n");
    }
    else
    {
        printf("Parent is waiting...\n");
        sleep(5);
        printf("Parent finished.\n");
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
```

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
hemasai@Hemasai:~/OSSP_Assignment$ nano Sigchild.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc Sigchild.c -o Sigchild
hemasai@Hemasai:~/OSSP_Assignment$ ./Sigchild
Parent is waiting...
Child is running...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.
hemasai@Hemasai:~/OSSP_Assignment$ |
```

Experiment 3 — SIGPIPE.

```
GNU nano 8.7.1 Si
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigpipe(int sig)
{
    printf("\nSIGPIPE received!\n");
    printf("The reading end of the pipe is closed.\n");
    exit(1);
}

int main()
{
    int fd[2];

    signal(SIGPIPE, handle_sigpipe);

    pipe(fd);

    printf("Pipe created.\n");

    close(fd[0]);

    printf("Reading end closed.\n");

    printf("Trying to write to the pipe...\n");

    write(fd[1], "Hello", 5);

    printf("Write completed.\n");

    return 0;
}
```

```
hemasai@Hemasai:~/OSSP_Assignment$ nano Sigchild.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc Sigchild.c -o Sigchild
hemasai@Hemasai:~/OSSP_Assignment$ ./Sigchild
Parent is waiting...
Child is running...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.
hemasai@Hemasai:~/OSSP_Assignment$ nano Sigpipe.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc Sigpipe.c -o Sigpipe
hemasai@Hemasai:~/OSSP_Assignment$ ./Sigpipe
Pipe created.
Reading end closed.
Trying to write to the pipe...

SIGPIPE received!
The reading end of the pipe is closed.
hemasai@Hemasai:~/OSSP_Assignment$ |
```

Experiment 4 — SIGSEGV

```

GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
    printf("\nSIGSEGV received!\n");
    printf("Invalid memory access detected.\n");
    exit(1);
}

int main()
{
    signal(SIGSEGV, handle_sigsegv);

    printf("Program started.\n");

    int *p = NULL;

    printf("Trying to access invalid memory...\n");

    *p = 10;

    printf("Program completed.\n");

    return 0;
}

```

```

The reading end of the pipe is closed.
hemasai@Hemasai:~/OSSP_Assignment$ nano Sigsegv.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc Sigsegv.c -o Sigsegv
hemasai@Hemasai:~/OSSP_Assignment$ ./Sigsegv
Program started.
Trying to access invalid memory...

SIGSEGV received!
Invalid memory access detected.
hemasai@Hemasai:~/OSSP_Assignment$ |

```